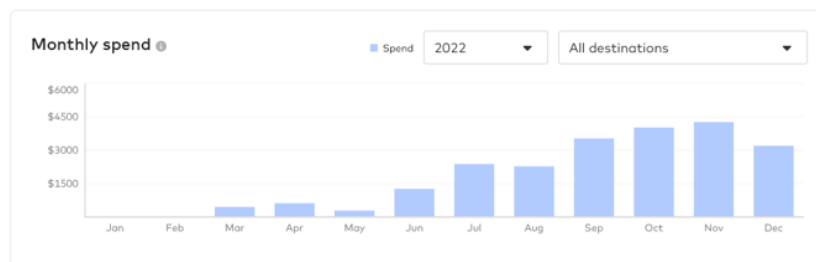
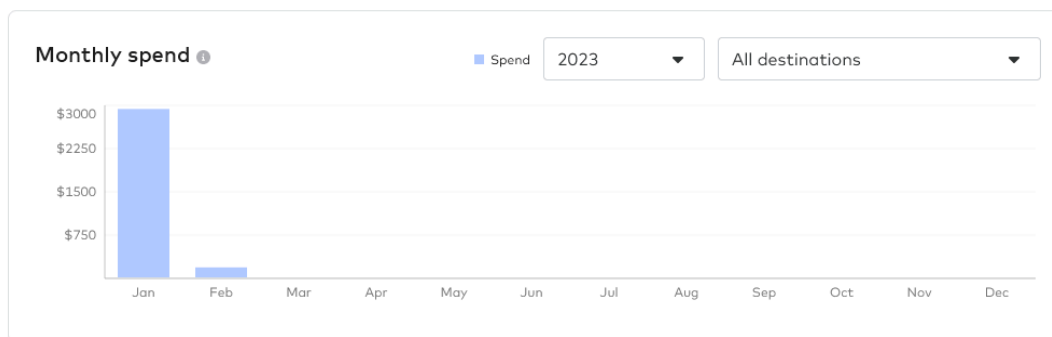


Fivetran cost evaluation

- [Cost](#)
 - [How cost is calculated](#)
 - [Where does our costs come from](#)
 - [Dev cost](#)
 - [Prod cost](#)
 - [Total costs](#)
 - [Monthly](#)
 - [Daily](#)
 - [Cost per table](#)
 - [Cost optimizations](#)
 - [Long term contract](#)
 - [Stripe workflow optimizations](#)
 - [Always free trial](#)
 - [Reduce the scope of the ingested data](#)
- [Value offered by Fivetran vs efforts to implement it ourselves](#)
 - [Incremental updates based on change data capture](#)
 - [Integration with many SaaS](#)
 - [Quick and easy to setup new sources](#)
 - [Has been running for many months without any manual interventions from the data team.](#)
 - [Data is structured and normalized before being sent to our databricks ecosystem](#)
 - [Fivetran alternatives](#)
- [Conclusion](#)
- [Recommendations](#)

Cost





How cost is calculated

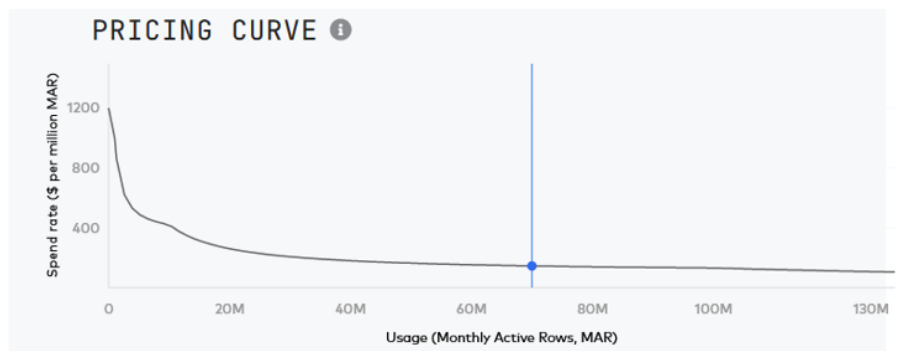
Cost is based on their concept of MAR or monthly active rows. When fivetran detects a change at the source for a row that it needs to sync then that row becomes active for a time period of a month. Subsequent updates to that row does not add to that MAR calculation as it is the same row that is being updated over and over.

Fivetran will charge by MAR used each month and try to reward its customers by giving better rate per MAR the more MAR you consume.

Example of pricing

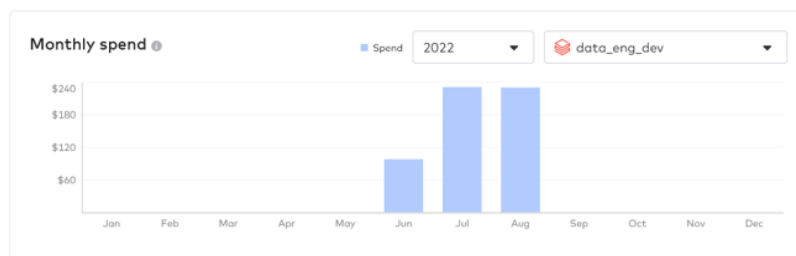
Monthly active rows	Price per million MAR (\$)	Price per month(\$)	Data volume growth	Total monthly cost growth	Cost per mar growth
1 Million	1000	1000	0	0	0
2 Million	701.75	1403.51	2	1.4	0.70175
7 million	451.13	3157.89	3.5	2.25	0.64
70 million	145.36	10175.44	10	3.2	0.32
98.8 million	131.97	13038.60	1.4	1.28	0.908
100 million +	Call for pricing information.				

For example going from 7 million to 70 is a 10x growth in data volume but would cost 3.2 times as much instead of 10x.



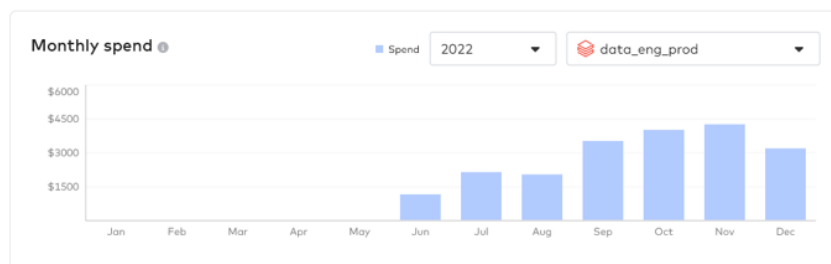
Where does our costs come from

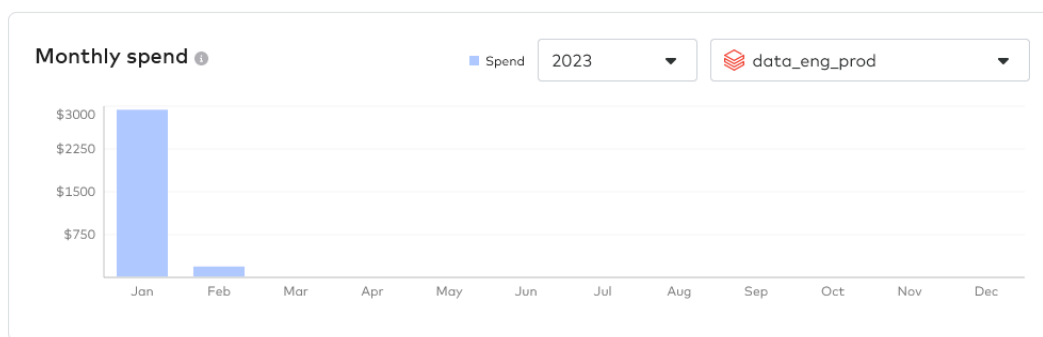
Dev cost



The volume of data from our dev environment is very low and it was not where our costs came from. However from our knowledge no one uses the dev data in the databricks environment currently so we paused the sync as it generated unnecessary costs on fivetran and databricks side.

Prod cost





Total costs

Monthly

Month	Paid MAR	MAR rate (\$/ M MAR)	Total spent (\$)
July	3,954,215	597.29	2,361.82
August	3,687,511	611.80	2,256.03
September	6,846,465	512.54	3,509.09
October	8,083,047	494.81	3,999.6
November	8,697,595	487.88	4,243.37
December	8,602,867	369.74	3,180.85
January	7,756,248	377.36	2,926.87
Total	47,627,948	3,451.42	22,477.63
Average	6,803,993	493.06	3,211.09

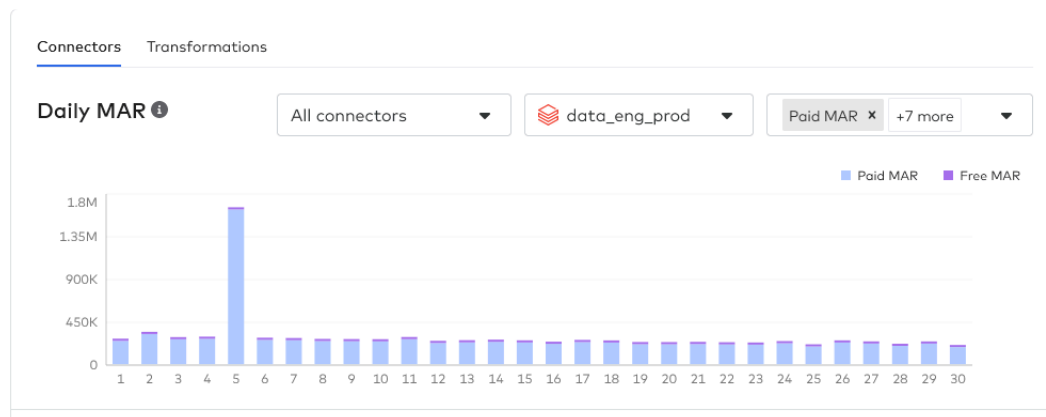
From the pricing curve we still have a lot of room that we can continue to get cheaper MAR as our consumption grows.



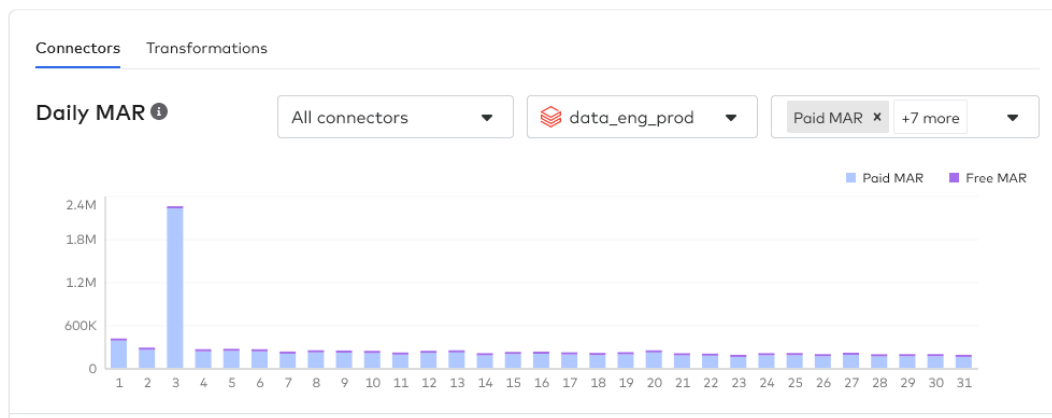
Our current MAR rate on the pricing curve

Daily

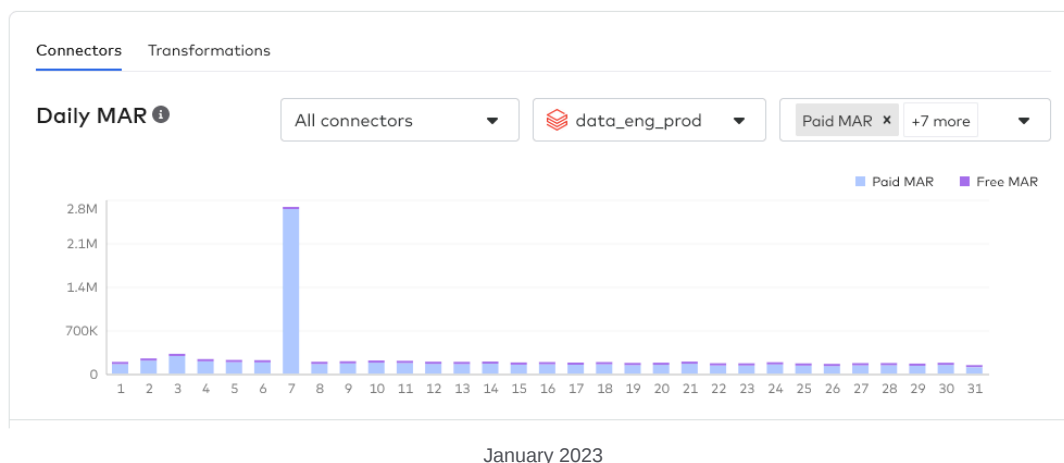
By looking at the daily costs we can see that there is one day per month that seems to contain most of the MAR. It represents the day that most of our MAR are refreshed.



November 2022



December 2022



Cost per table

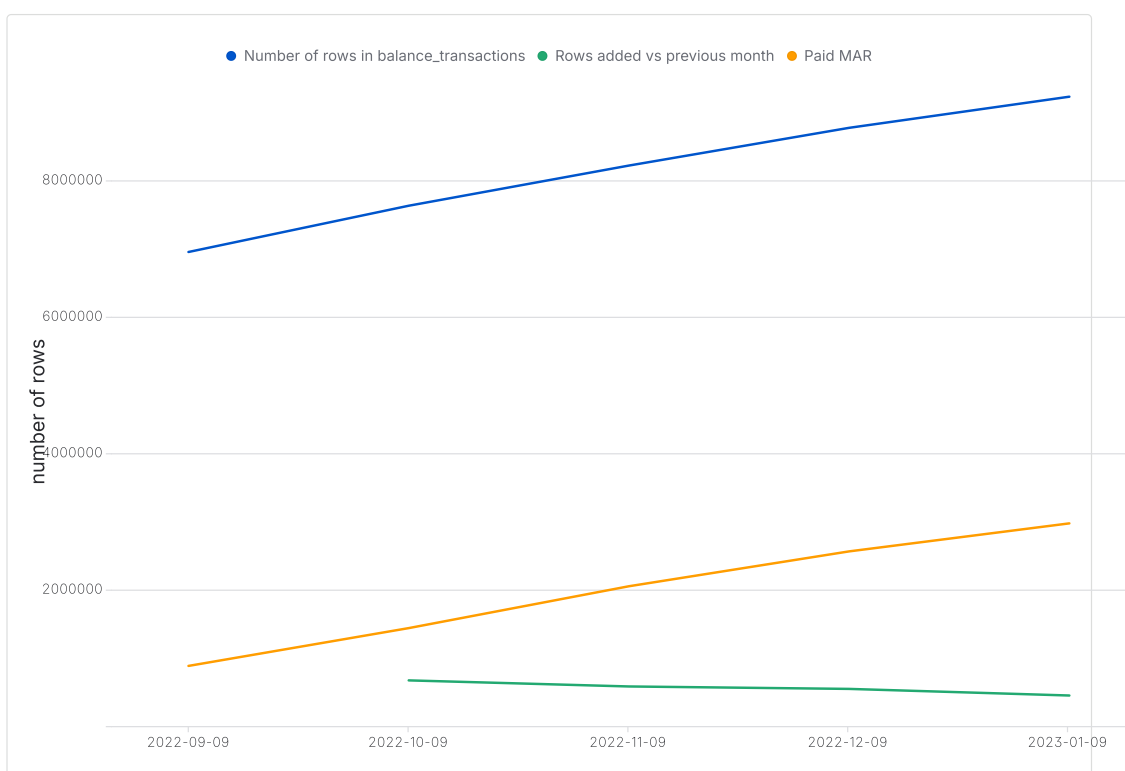
The number of MAR for the size of tables seem to be proportional to the size of the table we have within databricks. The distribution of the MAR is what we would expect since balance_transactions and subscription_history are our most voluminous tables.

Table	MAR ▼	% change ⓘ
balance_transaction	2,980,055	↑ 16 %
subscription_history	770,024	↓ 15 %
invoice_line_item	538,734	↓ 25 %
subscription_item	494,536	↓ 18 %
charge	455,731	↓ 24 %
payment_intent	418,252	↓ 26 %
invoice	397,137	↓ 25 %
fee	359,981	↓ 28 %
card	309,161	↓ 17 %
payment_method	291,083	↓ 14 %
invoice_item	145,270	↓ 35 %
payment_method_card	64,082	↑ 52 %
invoice_discount	59,749	↓ 18 %
customer_discount	51,503	↓ 17 %

Monthly active rows per table for January 2023

Number of transactions available within our bronze layers over time

Date	Number of rows in balance_transactions	Rows added vs previous month	Paid MAR
2022-09-09	6,958,134		890,138
2022-10-09	7,636,270	678,136	1,444,413
2022-11-09	8,225,265	588,995	2,058,127
2022-12-09	8,778,506	553,241	2,569,320
2023-01-09	9,234,929	456,423	2,980,055



We seem to have around 500,000 new transactions per month and about 2,500,000 monthly active rows which means around 2 million transactions are being updated month to month.

The growth of our Fivetran consumption scales directly based on the growth of our volume of data.



Cost optimizations

Long term contract

Currently the billing is on demand i believe further costs savings could be achieved by moving to a yearly contract. Back in September we did agree on a contract which came with costs saving and the impact was very apparent.

Stripe workflow optimizations

Most of our costs come from data that is being modified over the time span of more than 1 month so we are charged multiple times for the sync of the same rows.

I am still unsure if the transactions that are updated many times is caused by Stripe's workflow or it is due to how Spypoint interacts with stripe that causes a lot of rows to be updated multiple times over months.

Always free trial

We could have technically have 0 costs if we always create new connectors we get 2 week of free syncs and the initial sync is free so technically we could simply alternate between different copies of the data and roll through different connectors before the "trial" expires thus being free.

However i am unsure about business relationships with Fivetran and also it would be hard to have the data refreshed daily it would probably be a weekly refresh.

Reduce the scope of the ingested data

We could also re-verify with all the tables we ingest are they all required for the business needs if not removing them from the sync will lower costs.

Value offered by Fivetran vs efforts to implement it ourselves

Incremental updates based on change data capture

We would need to implement on our side a way to query the stripe API to only give us newly modified data. The effort to implement change data capture on data we have within our Databricks ecosystem is not that complicated however I am unsure how flexible the Stripe API is to be able to filter and only retrieve the fresh data.



Integration with many SaaS

Currently we only use Fivetran for Stripe so implementing the code to be able to extract the data from only one data source should not be complicated.

However that is more code that we have to maintain and this code will multiply quickly if we start to ingest other data sources that could have been ingested via Fivetran.

Quick and easy to setup new sources

Connecting to a new data source generally takes about 30 minutes. It is as simple as generated a API token or specifying login information in the Fivetran interface, select what data we want to sync and the sync will start.

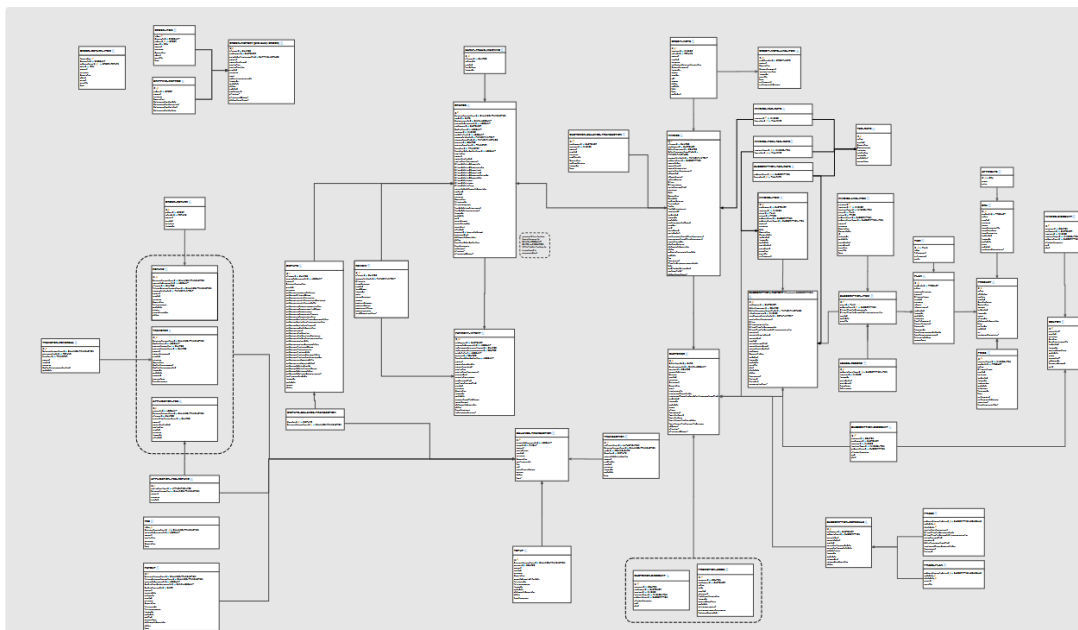
If we manually have to connect to a new data source we have to analyze what is the best way to connect to it to be able to extract the data. Then analyze the source format and determine if it fits the format that we would like it to have (normalized for example) and implement all the transformations ourselves.

Has been running for many months without any manual interventions from the data team.

Homemade code has to be maintained and updated periodically.

Data is structured and normalized before being sent to our databricks ecosystem

This is a big advantage that Fivetran offers. The ingested data is already normalized and a full schema is provided by Fivetran.



Stripe ERD offered by Fivetran

The source for stripe is different API has a different format than what we currently use via Fivetran. Instead of normalized tables we have a few JSON files that we would have to remodel to resemble what we already have.

Fivetran alternatives

Tools	Pricing model	Size limitations	CDC	Connectors with SaaS	Data normalization
Fivetran	130\$ to 450\$ per million row 8 million rows = 3,180.85 100 million rows = 13038.60	Not stated but need to call to know about pricing over 100m rows per month	CDC is based on comparing timestamps and only data after time xyz will be ingested.	275	Data is normalized into many different tables
Stitch advanced	Advanced : 1250\$/month (charged annually)	100 million rows/month		140+	No but documentation about the schema is given.
Stitch premium	2500 \$/month (charged annually)	1 billion rows/month			
Rivery	1 credit per sync per endpoint	1000gb per source?		200+ connectors	

	1 credit per workflow execution 4 Stripe accounts * 30 syncs/month = 120 credits * 1.2\$ = 144\$ / month			
Airbyte	6 credit per million rows 1 credit = 2.5\$ (depending on bulk purchases) 8 million rows * 6 credit = 160\$ 100 million rows * 6 credit = 1500\$	Not stated but they say that over 150gb or over 100 million rows need to talk to sales.		260+ (most of them are in alpha stage)

Based on this comparison we would save a lot of money if we would switch to another tool such as Rivery or Airbyte could save a few thousand dollars per month. Even Stitch which is quite more expensive for our current data load than Rivery or Airbyte would be a noticeable cost saving measure.

Functionality wise they are all similar in the features that they offer. The number of connectors available and how much is the data normalized is the two main differentiating factors.

Any of these tools that we would switch to a good amount of work would have to be put into relearning our data and how to regenerate the different KPI based on the new format of the data.

Conclusion

Fivetran has allowed us to quickly get started and ingest Stripe data with very little issues. Functionality wise we don't really have a reason to change tools or to redo internally the ETL for Stripe.

Our Fivetran costs scale directly with the volume of our data and we still have a lot of headroom to be able to ingest other data sources with the same tool. As we would

ramp up ingestion of other sources our cost per volume of data ingested will go down and make Fivetran seem a bit more competitive price wise.

However when we compare the cost of Fivetran for our current volume of data and with a data volume of 100 million rows per month Fivetran is 10x as more expensive than its competitors and that value is very hard to justify.

We also have the capacity of being able to implement such ELT process ourselves by querying the Stripe API directly and then do the normalization how we want to do it. A further study would have to be done about the feasibility but i believe 6 to 8 weeks of work would be sufficient to implement the ELT process. More time would then be needed to analyze the schema and come up with a data model.

Since we want to move from Stripe to another payment software like Bold should that amount of development be invested on Stripe when we can maybe just pay Fivetran a few more months until we pull the plug on Stripe?

Recommendations

If we are keeping Stripe long term then switching from Fivetran to a in house solution or to a cheaper competitor would bring significant cost savings.

If we plan on moving from Stripe to Bold in a few months than redoing the ELT process and the KPIs built on the data would probably not be worth the switch to another solution would probably not be as worth it.

Action items

- Seem to have a lot of rows active for the amount of new data
 - Need to validate on our end within stripe itself how much does the data change?
 - Does it seem to correspond to having nearly 2-3 million transactions updated monthly?
 - If not we need to validate with Fivetran how is the MAR calculated and why so much of the data is synced with no changes
- Meet with Fivetran and see what could be done pricing wise since competitors seem to be quite cheaper.
- Further research done into alternative sources with a proof of concept and contact the sales teams for proper cost evaluations.
- Do an analysis of all the further sources we would like to ingest and what tool best serve each data source.